




EIU
 TRƯỜNG ĐẠI HỌC
 QUỐC TẾ
 MIỀN ĐÔNG
 EASTERN
 INTERNATIONAL
 UNIVERSITY

CSE301 – DATABASE

SQL - Structured Query Language

Mobile programming

1

1



INTRODUCTION TO SQL

What is SQL?

- SQL stands for Structured Query Language
- SQL lets you access and manipulate databases

What Can SQL do?

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views.



2

2



SQL CREATE DATABASE STATEMENT

The CREATE DATABASE statement is used to create a database.

SQL CREATE DATABASE Syntax: *CREATE DATABASE database_name*

CREATE DATABASE Example

Now we want to create a database called "my_db". We use the following

CREATE DATABASE statement:

CREATE DATABASE my_db

3

3



SQL CREATE TABLE STATEMENT

In relational database systems (DBS) data are represented using tables(relations).

The CREATE TABLE statement is used to create a table in a database.

SQL CREATE TABLE Syntax

```
CREATE TABLE table_name (
column_name1 data_type,
column_name2 data_type,
column_name3 data_type,
....
)
```

4

4



SQL CREATE TABLE STATEMENT

CREATE TABLE Example

Now we want to create a table called "Persons" that contains five columns: P_Id, LastName, FirstName, Address, and City.

```
CREATE TABLE Persons (
P_Id int,
LastName varchar(255), FirstName varchar(255),
Address varchar(255), City varchar(255)
)
```

P_Id	LastName	FirstName	Address	City

5

5



MYSQL DATA TYPES

In MySQL there are three main types : text, number, and Date/Time types.

Text types:

Data type	Description
CHAR(size)	Holds a fixed length string (can contain letters, numbers, and special characters). The fixed size is specified in parenthesis. Can store up to 255 characters
VARCHAR(size)	Holds a variable length string (can contain letters, numbers, and special characters). The maximum size is specified in parenthesis. Can store up to 255 characters. Note: If you put a greater value than 255 it will be converted to a TEXT type
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT	Holds a string with a maximum length of 65,535 characters
BLOB	For BLOBs (Binary Large Objects). Holds up to 65,535 bytes of data
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
MEDIUMBLOB	For BLOBs (Binary Large Objects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LONGBLOB	For BLOBs (Binary Large Objects). Holds up to 4,294,967,295 bytes of data
ENUM(x,y,z,etc.)	Let you enter a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. Note: The values are sorted in the order you enter them. You enter the possible values in this format: ENUM('X','Y','Z')
SET	Similar to ENUM except that SET may contain up to 64 list items and can store more than one choice

6

6



MYSQL DATA TYPES

Number types:

Data type	Description
TINYINT(size)	-128 to 127 normal. 0 to 255 UNSIGNED*. The maximum number of digits may be specified in parenthesis
SMALLINT(size)	-32768 to 32767 normal. 0 to 65535 UNSIGNED*. The maximum number of digits may be specified in parenthesis
MEDIUMINT(size)	-8388608 to 8388607 normal. 0 to 16777215 UNSIGNED*. The maximum number of digits may be specified in parenthesis
INT(size)	-2147483648 to 2147483647 normal. 0 to 4294967295 UNSIGNED*. The maximum number of digits may be specified in parenthesis
BIGINT(size)	-9223372036854775808 to 9223372036854775807 normal. 0 to 18446744073709551615 UNSIGNED*. The maximum number of digits may be specified in parenthesis
FLOAT(size,d)	A small number with a floating decimal point. The maximum number of digits may be specified in the size parameter. The maximum number of digits to the right of the decimal point is specified in the d parameter
DOUBLE(size,d)	A large number with a floating decimal point. The maximum number of digits may be specified in the size parameter. The maximum number of digits to the right of the decimal point is specified in the d parameter
DECIMAL(size,d)	A DOUBLE stored as a string, allowing for a fixed decimal point. The maximum number of digits may be specified in the size parameter. The maximum number of digits to the right of the decimal point is specified in the d parameter

7

7



MYSQL DATA TYPES

Date types:

Data type	Description
DATE()	A date. Format: YYYY-MM-DD Note: The supported range is from '1000-01-01' to '9999-12-31'
DATETIME()	*A date and time combination. Format: YYYY-MM-DD HH:MM:SS Note: The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'
TIMESTAMP()	*A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD HH:MM:SS Note: The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC
TIME()	A time. Format: HH:MM:SS Note: The supported range is from '-838:59:59' to '838:59:59'
YEAR()	A year in two-digit or four-digit format.

8

8



SQL INSERT INTO STATEMENT

The INSERT INTO statement is used to insert a new row in a table.

SQL INSERT INTO Syntax

INSERT INTO table_name

VALUES (value1, value2, value3,...)

The second form specifies both the column names and the values to be inserted:

INSERT INTO table_name (column1, column2, column3,...)

VALUES (value1, value2, value3,...)

9

9



SQL INSERT INTO STATEMENT

SQL INSERT INTO Example

We have the following "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

```
INSERT INTO Persons
VALUES (4,'Nilsen', 'Johan', 'Bakken 2', 'Stavanger')
```

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger

10

10



SQL INSERT INTO STATEMENT

SQL INSERT INTO Example

Insert Data Only in Specified Columns

INSERT INTO Persons (P_Id, LastName, FirstName)

VALUES (5, 'Tjessem', 'Jakob')

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jakob		

11

11



SQL CONSTRAINTS

Constraints are used to limit the type of data that can go into a table. Constraints can be specified when a table is created (with the CREATE TABLE statement) or after the table is created (with the ALTER TABLE statement).

We will focus on the following constraints:

- NOT NULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY
- CHECK
- DEFAULT

12

12



SQL NOT NULL CONSTRAINT

By default, a table column can hold NULL values.

The NOT NULL constraint enforces a column to NOT accept NULL values.

The following SQL enforces the "P_Id" column and the "LastName" column to not accept NULL values:

Example:

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255)
)
```

13

13



SQL UNIQUE CONSTRAINT

The UNIQUE constraint uniquely identifies each record in a database table.

The UNIQUE and PRIMARY KEY constraints both provide a guarantee for uniqueness for a column or set of columns.

Example:

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  UNIQUE (P_Id)
)
```

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  CONSTRAINT uc_PersonID UNIQUE (P_Id,LastName)
)
```

14

14



SQL UNIQUE Constraint on ALTER TABLE

To create a UNIQUE constraint on the "P_Id" column when the table is already created, use the following SQL:

Example: *ALTER TABLE Persons ADD UNIQUE (P_Id)*

ALTER TABLE Persons ADD CONSTRAINT uc_PersonID UNIQUE (P_Id,LastName)

To DROP a UNIQUE Constraint

ALTER TABLE Persons DROP INDEX uc_PersonID

15

15



SQL PRIMARY KEY Constraint

- The PRIMARY KEY constraint uniquely identifies each record in a database table.
- Primary keys must contain unique values.
- A primary key column cannot contain NULL values.
- Each table should have a primary key, and each table can have only ONE primary key.

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  PRIMARY KEY (P_Id)
)
```

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  CONSTRAINT pk_PersonID PRIMARY KEY (P_Id,LastName)
)
```

16

16



SQL PRIMARY KEY Constraint on ALTER TABLE

To create a PRIMARY KEY constraint on the "P_Id" column when the table is already created, use the following SQL

```
ALTER TABLE Persons ADD PRIMARY KEY (P_Id)
```

```
ALTER TABLE Persons
```

```
ADD CONSTRAINT pk_PersonID PRIMARY KEY (P_Id, LastName)
```

To DROP a PRIMARY KEY Constraint

```
ALTER TABLE Persons DROP PRIMARY KEY
```

```
ALTER TABLE Persons
```

```
DROP CONSTRAINT pk_PersonID
```

17

17



SQL FOREIGN KEY Constraint

A FOREIGN KEY in one table points to a PRIMARY KEY in another table.

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

The "Orders" table:

O_Id	OrderNo	P_Id
1	77895	3
2	44678	3
3	22456	2
4	24562	1

18

18



SQL FOREIGN KEY Constraint on CREATE TABLE

The following SQL creates a FOREIGN KEY on the "P_Id" column when the "Orders" table is created:

```
CREATE TABLE Orders
(
  O_Id int NOT NULL,
  OrderNo int NOT NULL,
  P_Id int,
  PRIMARY KEY (O_Id),
  FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)
)
```

```
CREATE TABLE Orders
(
  O_Id int NOT NULL,
  OrderNo int NOT NULL,
  P_Id int,
  PRIMARY KEY (O_Id),
  CONSTRAINT fk_PerOrders FOREIGN KEY (P_Id)
  REFERENCES Persons(P_Id)
)
```

19

19



SQL FOREIGN KEY Constraint on ALTER TABLE

To create a FOREIGN KEY constraint on the "P_Id" column when the "Orders" table is already created, use the following SQL

```
ALTER TABLE Orders ADD FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)
```

```
ALTER TABLE Orders ADD CONSTRAINT fk_PerOrders FOREIGN KEY (P_Id)
REFERENCES Persons(P_Id)
```

To DROP a FOREIGN KEY Constraint

```
ALTER TABLE Orders DROP FOREIGN KEY fk_PerOrders
```

20

20



SQL CHECK Constraint

The CHECK constraint is used to limit the value range that can be placed in a column

SQL CHECK Constraint on CREATE TABLE

```
CREATE TABLE Persons
```

```
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL, FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  CHECK (P_Id>0)
)
```

```
CREATE TABLE Persons
```

```
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  CONSTRAINT chk_Person CHECK (P_Id>0 AND City='Sandnes')
)
```

21

21



SQL CHECK Constraint

SQL CHECK Constraint on ALTER TABLE

```
ALTER TABLE Persons ADD CHECK (P_Id>0)
```

```
ALTER TABLE Persons ADD CONSTRAINT chk_Person CHECK (P_Id>0 AND
City='Sandnes')
```

To DROP a CHECK Constraint

```
ALTER TABLE Persons DROP CONSTRAINT chk_Person
```

22

22



SQL DEFAULT Constraint

The DEFAULT constraint is used to insert a default value into a column.

SQL DEFAULT Constraint on CREATE TABLE

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255) DEFAULT 'Sandnes'
)
```

```
CREATE TABLE Orders
(
  O_Id int NOT NULL,
  OrderNo int NOT NULL,
  P_Id int,
  OrderDate date DEFAULT GETDATE()
)
```

23

23



SQL DEFAULT Constraint

SQL DEFAULT Constraint on ALTER TABLE

ALTER TABLE Persons ALTER City SET DEFAULT 'SANDNES'

To DROP a DEFAULT Constraint

ALTER TABLE Persons ALTER City DROP DEFAULT

24

24



SQL SYNTAX

A database most often contains one or more tables. Each table is identified by a name (e.g. "Customers" or "Orders"). Tables contain records (rows) with data. Below is an example of a table called "Persons":

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

25

25



SQL SYNTAX

SQL DML and DDL

The query and update commands form the DML part of SQL:

- **SELECT** - extracts data from a database
- **UPDATE** - updates data in a database
- **DELETE** - deletes data from a database
- **INSERT INTO** - inserts new data into a database

The DDL part of SQL permits database tables to be created or deleted. It also define indexes (keys), specify links between tables, and impose constraints between tables. The most important DDL statements in SQL are:

- **CREATE DATABASE** - creates a new database
- **ALTER DATABASE** - modifies a database
- **CREATE TABLE** - creates a new table
- **ALTER TABLE** - modifies a table
- **DROP TABLE** - deletes a table
- **CREATE INDEX** - creates an index (search key)
- **DROP INDEX** - deletes an index

26

26



SQL SELECT STATEMENT

The SELECT statement is used to select data from a database. The result is stored in a result table, called the result-set.

Most of the actions you need to perform on a database are done with SQL statements.

The following SQL statement will select all the records in the "Persons" table:

SQL SELECT Syntax

SELECT column_name(s) FROM table_name

*SELECT * FROM table_name*

27

27



SQL SELECT STATEMENT

SQL SELECT Example

*SELECT * FROM Persons*

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

SELECT LastName,FirstName FROM Persons

LastName	FirstName
Hansen	Ola
Svendson	Tove
Pettersen	Kari

28

28



SQL SELECT STATEMENT

SQL SELECT DISTINCT STATEMENT

In a table, some of the columns may contain duplicate values. This is not a problem, however, sometimes you will want to list only the different (distinct) values in a table. The DISTINCT keyword can be used to return only distinct (different) values.

SQL SELECT DISTINCT Syntax.

SELECT DISTINCT column_name(s) FROM table_name

SELECT DISTINCT Example

SELECT DISTINCT City FROM Persons

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger



City
Sandnes
Stavanger

29

29



SQL SELECT STATEMENT

SQL SELECT DISTINCT STATEMENT

In a table, some of the columns may contain duplicate values. This is not a problem, however, sometimes you will want to list only the different (distinct) values in a table. The DISTINCT keyword can be used to return only distinct (different) values.

SQL SELECT DISTINCT Syntax.

SELECT DISTINCT column_name(s) FROM table_name

SELECT DISTINCT Example

SELECT DISTINCT City FROM Persons

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger



City
Sandnes
Stavanger

30

30



SQL SELECT STATEMENT

SQL WHERE CLAUSE

The WHERE clause is used to filter records.

SQL WHERE Syntax

SELECT column_name(s)

FROM table_name

WHERE column_name operator value

WHERE Clause Example

*SELECT * FROM Persons WHERE City='Sandnes'*

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes

31

31



SQL SELECT STATEMENT

Operators Allowed in the WHERE Clause

Operator	Description
=	Equal
<>	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
BETWEEN	Between an inclusive range
LIKE	Search for a pattern
IN	If you know the exact <u>value</u> you want to return for at least one of the columns

32

32



SQL SELECT STATEMENT

SQL AND & OR OPERATORS

The AND operator displays a record if both the first condition and the second condition is true.
The OR operator displays a record if either the first condition or the second condition is true.

AND Operator Example

*SELECT * FROM Persons WHERE FirstName='Tove' AND LastName='Svendson'*

P_Id	LastName	FirstName	Address	City
2	Svendson	Tove	Borgvn 23	Sandnes

OR Operator Example

*SELECT * FROM Persons WHERE FirstName='Tove' OR FirstName='Ola'*

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes

33

33



SQL SELECT STATEMENT

SQL ORDER BY KEYWORD

The ORDER BY keyword is used to sort the result-set.

SQL ORDER BY Syntax

SELECT column_name(s)

FROM table_name

ORDER BY column_name(s) ASC/DESC

ORDER BY Example

*SELECT * FROM Persons ORDER BY LastName*

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
4	Nilsen	Tom	Vingvn 23	Stavanger
3	Pettersen	Kari	Storgt 20	Stavanger
2	Svendson	Tove	Borgvn 23	Sandnes

34

34



SQL SELECT STATEMENT

ORDER BY DESC Example

*SELECT * FROM Persons ORDER BY LastName DESC*

P_Id	LastName	FirstName	Address	City
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Tom	Vingvn 23	Stavanger
1	Hansen	Ola	Timoteivn 10	Sandnes

35

35



SQL UPDATE STATEMENT

The UPDATE statement is used to update existing records in a table.

SQL UPDATE Syntax

UPDATE table_name

SET column1=value, column2=value2,...

WHERE some_column=some_value

SQL UPDATE Example

UPDATE Persons

SET Address='Nissestien 67', City='Sandnes'

WHERE LastName='Tjessem' AND FirstName='Jakob'

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jakob		



Address	City
Timoteivn 10	Sandnes
Borgvn 23	Sandnes
Storgt 20	Stavanger
Bakken 2	Stavanger
Nissestien 67	Sandnes

36

36



SQL DELETE STATEMENT

The DELETE statement is used to delete records in a table.

SQL DELETE Syntax

DELETE FROM *table_name*
WHERE *some_column=some_value*

SQL DELETE Example

DELETE FROM Persons
WHERE LastName='Tjessem' AND FirstName='Jakob'

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jakob	Nissestien 67	Sandnes



P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger

37

37



SQL TOP CLAUSE

The TOP clause is used to specify the number of records to return.

The TOP clause can be very useful on large tables with thousands of records. Returning a large number of records can impact on performance.

SQL Server Syntax

SELECT TOP *number/percent column_name(s)*
FROM *table_name*

SQL TOP Example

SELECT TOP 2 * *FROM* Persons

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jakob	Nissestien 67	Sandnes



P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes

38

38



SQL TOP CLAUSE

SQL TOP PERCENT Example

Now we want to select only 50% of the records in the table above.

We use the following SELECT statement:

***SELECT TOP 50 PERCENT *
FROM Persons***

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Tom	Vingvn 23	Stavanger



P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes

39

39



SQL LIKE OPERATOR

SQL WILDCARDS

SQL wildcards can be used when searching for data in a database.

SQL wildcards must be used with the SQL LIKE operator.

Wildcard	Description
%	A substitute for zero or more characters
_	A substitute for exactly one character
[charlist]	Any single character in charlist
[^charlist]	Any single character not in charlist
or	
[!charlist]	

40

40



SQL LIKE OPERATOR

SQL WILDCARDS

SQL Wildcard Examples

*SELECT * FROM Persons*
WHERE FirstName LIKE '_la'



P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes

*SELECT * FROM Persons*
WHERE LastName LIKE '[bsp]%'



P_Id	LastName	FirstName	Address	City
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

*SELECT * FROM Persons*
WHERE LastName LIKE '[!bsp]%'



P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes

41

41



SQL IN OPERATOR

The IN operator allows you to specify multiple values in a WHERE clause.

SQL IN Syntax

SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1,value2,...)

IN Operator Example

*SELECT * FROM Persons*
WHERE LastName IN ('Hansen','Pettersen')

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger



P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

42

42



SQL BETWEEN OPERATOR

The BETWEEN operator is used in a WHERE clause to select a range of data between two values.

SQL BETWEEN Syntax

SELECT column_name(s)

FROM table_name

WHERE column_name BETWEEN value1 AND value2

BETWEEN Operator Example

*SELECT * FROM Persons*

WHERE LastName

BETWEEN 'Hansen' AND 'Pettersen'

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger



P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes

43

43



SQL BETWEEN OPERATOR

To display the persons outside the range in the previous example, use NOT BETWEEN:

*SELECT **

FROM Persons

WHERE LastName NOT BETWEEN 'Hansen' AND 'Pettersen'

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger



P_Id	LastName	FirstName	Address	City
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

44

44



SQL ALIAS

With SQL, an alias name can be given to a table or to a column.

SQL Alias Syntax for Tables

```
SELECT column_name(s)
FROM table_name AS alias_name
```

SQL Alias Syntax for Columns

```
SELECT column_name AS alias_name
FROM table_name
```

Alias Example

```
SELECT po.OrderID, p.LastName, p.FirstName FROM Persons AS p,
Product_Orders AS po
WHERE p.LastName='Hansen' AND p.FirstName='Ola'
```

45

45



SQL JOINS

SQL joins are used to query data from two or more tables, based on a relationship between certain columns in these tables.

The "Persons" table

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

The "Orders" table

O_Id	OrderNo	P_Id
1	77895	3
2	44678	3
3	22456	1
4	24562	1
5	34764	15

46

46



SQL JOINS

Different SQL JOINS

JOIN: Return rows when there is at least one match in both tables

LEFT JOIN: Return all rows from the left table, even if there are no matches in the right table

RIGHT JOIN: Return all rows from the right table, even if there are no matches in the left table

FULL JOIN: Return rows when there is a match in one of the tables

47

47



SQL INNER JOIN

The INNER JOIN keyword return rows when there is at least one match in both tables.

SQL INNER JOIN Syntax

```
SELECT column_name(s)
FROM table_name1 INNER JOIN table_name2
ON table_name1.column_name=table_name2.column_name
```

SQL INNER JOIN Example

```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo FROM Persons
INNER JOIN Orders ON Persons.P_Id=Orders.P_Id ORDER BY Persons.LastName
```

48

48



SQL INNER JOIN

SQL INNER JOIN Example

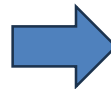
The "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

The "Orders" table:

O_Id	OrderNo	P_Id
1	77895	3
2	44678	3
3	22456	1
4	24562	1
5	34764	15

```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo FROM Persons
INNER JOIN Orders
ON Persons.P_Id=Orders.P_Id ORDER BY Persons.LastName
```



LastName	FirstName	OrderNo
Hansen	Ola	22456
Hansen	Ola	24562
Pettersen	Kari	77895
Pettersen	Kari	44678

49

49



SQL LEFT JOIN

The LEFT JOIN keyword returns all rows from the left table (table_name1), even if there are no matches in the right table (table_name2).

SQL LEFT JOIN Syntax

```
SELECT column_name(s)
FROM table_name1 LEFT JOIN table_name2
ON table_name1.column_name=table_name2.column_name
```

SQL LEFT JOIN Example

```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo FROM Persons
LEFT JOIN Orders
ON Persons.P_Id=Orders.P_Id ORDER BY Persons.LastName
```

50

50



SQL LEFT JOIN

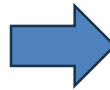
SQL LEFT JOIN Example

The "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

The "Orders" table:

O_Id	OrderNo	P_Id
1	77895	3
2	44678	3
3	22456	1
4	24562	1
5	34764	15



```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo FROM Persons
LEFT JOIN Orders
ON Persons.P_Id=Orders.P_Id ORDER BY Persons.LastName
```

LastName	FirstName	OrderNo
Hansen	Ola	22456
Hansen	Ola	24562
Pettersen	Kari	77895
Pettersen	Kari	44678
Svendson	Tove	

51

51



SQL RIGHT JOIN

The RIGHT JOIN keyword Return all rows from the right table (table_name2), even if there are no matches in the left table (table_name1).

SQL RIGHT JOIN Syntax

```
SELECT column_name(s)
FROM table_name1 RIGHT JOIN table_name2
ON table_name1.column_name=table_name2.column_name
```

SQL RIGHT JOIN Example

```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo
FROM Persons RIGHT JOIN Orders
ON Persons.P_Id=Orders.P_Id ORDER BY Persons.LastName
```

52

52



SQL RIGHT JOIN

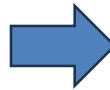
SQL RIGHT JOIN Example

The "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

The "Orders" table:

O_Id	OrderNo	P_Id
1	77895	3
2	44678	3
3	22456	1
4	24562	1
5	34764	15



```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo
FROM Persons RIGHT JOIN Orders
ON Persons.P_Id=Orders.P_Id ORDER BY Persons.LastName
```

LastName	FirstName	OrderNo
Hansen	Ola	22456
Hansen	Ola	24562
Pettersen	Kari	77895
Pettersen	Kari	44678
		34764

53

53



SQL FULL JOIN

The FULL JOIN keyword return rows when there is a match in one of the tables.

SQL FULL JOIN Syntax

```
SELECT column_name(s)
FROM table_name1 FULL JOIN table_name2
ON table_name1.column_name=table_name2.column_name
```

SQL FULL JOIN Example

```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo
FROM Persons FULL JOIN Orders
ON Persons.P_Id=Orders.P_Id ORDER BY Persons.LastName
```

54

54



SQL FULL JOIN

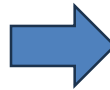
SQL FULL JOIN Example

The "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

The "Orders" table:

O_Id	OrderNo	P_Id
1	77895	3
2	44678	3
3	22456	1
4	24562	1
5	34764	15



LastName	FirstName	OrderNo
Hansen	Ola	22456
Hansen	Ola	24562
Pettersen	Kari	77895
Pettersen	Kari	44678
Svendson	Tove	
		34764

55

55



SQL UNION OPERATOR

The UNION operator is used to combine the result-set of two or more SELECT statements.

SQL UNION Syntax

```
SELECT column_name(s) FROM table_name1
UNION
SELECT column_name(s) FROM table_name2
```

Note: The column names in the result-set of a UNION are always equal to the column names in the first SELECT statement in the UNION.

56

56



SQL UNION OPERATOR

SQL UNION Example

"Employees_Norway":

E_ID	E_Name
01	Hansen, Ola
02	Svendson, Tove
03	Svendson, Stephen
04	Pettersen, Kari

"Employees_USA":

E_ID	E_Name
01	Turner, Sally
02	Kent, Clark
03	Svendson, Stephen
04	Scott, Stephen

57

57



SQL UNION OPERATOR

SQL UNION Example

```
SELECT E_Name FROM Employees_Norway
UNION
SELECT E_Name FROM Employees_USA
```

Note: This command cannot be used to list all employees in Norway and USA. In the example above we have two employees with equal names, and only one of them will be listed. The UNION command selects only distinct values.

E_Name
Hansen, Ola
Svendson, Tove
Svendson, Stephen
Pettersen, Kari
Turner, Sally
Kent, Clark
Scott, Stephen

58

58



SQL UNION OPERATOR

SQL UNION **ALL** Example

```
SELECT E_Name FROM Employees_Norway
UNION ALL
SELECT E_Name FROM Employees_USA
```

E_Name	
Hansen, Ola	
Svendson, Tove	
Svendson, Stephen	
Pettersen, Kari	
Turner, Sally	
Kent, Clark	
Svendson, Stephen	
Scott, Stephen	

59

59



SQL SELECT INTO STATEMENT

The SELECT INTO statement selects data from one table and inserts it into a different table. The SELECT INTO statement is most often used to create backup copies of tables.

SQL SELECT INTO Syntax

```
SELECT * INTO new_table_name [IN externaldatabase]
FROM old_tablename
```

```
SELECT column_name(s) INTO new_table_name [IN externaldatabase]
FROM old_tablename
```

60

60



SQL SELECT INTO STATEMENT

SQL SELECT INTO Example

```
SELECT * INTO Persons_Backup
FROM Persons
```

We can also use the IN clause to copy the table into another database:

```
SELECT * INTO Persons_Backup IN 'Backup.mdb'
FROM Persons
```

We can also copy only a few fields into the new table:

```
SELECT LastName,FirstName INTO Persons_Backup
FROM Persons
```

61

61



SQL SELECT INTO STATEMENT

SQL SELECT INTO - With a WHERE Clause

```
SELECT LastName,Firstname INTO Persons_Backup
FROM Persons
WHERE City='Sandnes'
```

SQL SELECT INTO - Joined Tables

```
SELECT Persons.LastName,Orders.OrderNo INTO Persons_Order_Backup
FROM Persons INNER JOIN Orders
ON Persons.P_Id=Orders.P_Id
```

62

62



SQL CREATE INDEX STATEMENT

The CREATE INDEX statement is used to create indexes in tables
 An index can be created in a table to find data more quickly and efficiently.
 The users cannot see the indexes, they are just used to speed up searches/queries.

SQL CREATE INDEX Syntax

Creates an index on a table. Duplicate values are allowed:

```
CREATE INDEX index_name ON table_name (column_name)
```

SQL CREATE UNIQUE INDEX Syntax

Creates a unique index on a table. Duplicate values are not allowed:

```
CREATE UNIQUE INDEX index_name ON table_name (column_name)
```

63

63



SQL CREATE INDEX STATEMENT

CREATE INDEX Example

```
CREATE INDEX Pindex ON Persons (LastName)
```

```
CREATE INDEX Pindex ON Persons (LastName, FirstName)
```

64

64



SQL DROP INDEX, DROP TABLE, and DROP DATABASE

The DROP INDEX Statement

ALTER TABLE table_name DROP INDEX index_name

The DROP TABLE Statement

DROP TABLE table_name

The DROP DATABASE Statement

DROP DATABASE database_name

65

65



SQL ALTER TABLE STATEMENT

SQL ALTER TABLE Syntax

To add a column in a table, use the following syntax:

ALTER TABLE table_name ADD column_name datatype

To delete a column in a table, use the following syntax :

ALTER TABLE table_name DROP COLUMN column_name

To change the data type of a column in a table, use the following syntax:

ALTER TABLE table_name ALTER COLUMN column_name datatype

66

66



SQL ALTER TABLE STATEMENT

SQL ALTER TABLE Example

*ALTER TABLE Persons **ADD DateOfBirth date***

Change Data Type Example

*ALTER TABLE Persons **ALTER COLUMN DateOfBirth year***

DROP COLUMN Example

*ALTER TABLE Persons **DROP COLUMN DateOfBirth***

67

67



SQL AUTO INCREMENT FIELD

The following SQL statement defines the "P_Id" column to be **an auto-increment primary key** field in the "Persons" table:

```
CREATE TABLE Persons (  
  P_Id int NOT NULL AUTO_INCREMENT,  
  LastName varchar(255) NOT NULL, FirstName varchar(255),  
  Address varchar(255),  
  City varchar(255), PRIMARY KEY (P_Id)  
)
```

The starting value for AUTO_INCREMENT is 1, and it will increment by 1 for each new record.

*ALTER TABLE Persons **AUTO_INCREMENT=100** (Start)*

68

68



SQL VIEWS

A view is a virtual table.

SQL CREATE VIEW Statement

A view is a virtual table based on the result-set of an SQL statement. A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.

SQL CREATE VIEW Syntax

```
CREATE VIEW view_name
```

```
AS
```

```
SELECT column_name(s) FROM table_name
```

```
WHERE condition
```

69

69



SQL VIEWS

SQL CREATE VIEW Examples

```
CREATE VIEW [Current Product List]
```

```
AS
```

```
SELECT ProductID,ProductName
```

```
FROM Products
```

```
WHERE Discontinued=No
```

We can query the view:

```
SELECT * FROM [Current Product List]
```

70

70



SQL VIEWS

SQL CREATE VIEW Examples

```
CREATE VIEW [Products Above Average Price]
AS
SELECT ProductName,UnitPrice
FROM Products
WHERE UnitPrice>(SELECT AVG(UnitPrice) FROM Products)
```

We can query the view:

```
SELECT * FROM [Products Above Average Price]
```

71

71



SQL VIEWS

SQL Updating a View

You can update a view by using the following syntax:

```
CREATE OR REPLACE VIEW view_name
AS
SELECT column_name(s)
FROM table_name WHERE condition
```

Example

```
CREATE VIEW [Current Product List]
AS
SELECT ProductID,ProductName,Category FROM Products
WHERE Discontinued=No
```

72

72



SQL VIEWS

SQL Dropping a View

SQL DROP VIEW Syntax

DROP VIEW *view_name*

73

73



SQL DATE FUNCTIONS

Function	Description
NOW()	Returns the current date and time
CURDATE()	Returns the current date
CURTIME()	Returns the current time
DATE()	Extracts the date part of a date or date/time expression
EXTRACT()	Returns a single part of a date/time
DATE_ADD()	Adds a specified time interval to a date
DATE_SUB()	Subtracts a specified time interval from a date
DATEDIFF()	Returns the number of days between two dates
DATE_FORMAT()	Displays date/time data in different formats

74

74



SQL NULL VALUES

NULL values represent missing unknown data. By default, a table column can hold NULL values

SQL IS NULL

```
SELECT LastName,FirstName,Address
FROM Persons
WHERE Address IS NULL
```

SQL IS NOT NULL

```
SELECT LastName,FirstName,Address
FROM Persons
WHERE Address IS NOT NULL
```

75

75



SQL FUNCTIONS

SQL Aggregate Functions

- AVG() - Returns the average value
- COUNT() - Returns the number of rows
- FIRST() - Returns the first value
- LAST() - Returns the last value
- MAX() - Returns the largest value
- MIN() - Returns the smallest value
- SUM() - Returns the sum

SQL Scalar functions

- UCASE() - Converts a field to upper case
- LCASE() - Converts a field to lower case
- MID() - Extract characters from a text field
- LEN() - Returns the length of a text field
- ROUND() - Rounds a numeric field to the number of decimals specified
- NOW() - Returns the current system date and time
- FORMAT() - Formats how a field is to be displayed

76

76



SQL FUNCTIONS

SQL AVG() Function

SELECT AVG(column_name) FROM table_name

SQL AVG() Example

*SELECT AVG(OrderPrice) AS OrderAverage
FROM Orders*

O_Id	OrderDate	OrderPrice	Customer
1	2008/11/12	1000	Hansen
2	2008/10/23	1600	Nilsen
3	2008/09/02	700	Hansen
4	2008/09/03	300	Hansen
5	2008/08/30	2000	Jensen
6	2008/10/04	100	Nilsen



OrderAverage
950

77

77



SQL FUNCTIONS

SQL COUNT() Function

SELECT COUNT(column_name) FROM table_name

SQL COUNT(column_name) Example

*SELECT COUNT(Customer) AS CustomerNilsen
FROM Orders WHERE Customer='Nilsen'*

O_Id	OrderDate	OrderPrice	Customer
1	2008/11/12	1000	Hansen
2	2008/10/23	1600	Nilsen
3	2008/09/02	700	Hansen
4	2008/09/03	300	Hansen
5	2008/08/30	2000	Jensen
6	2008/10/04	100	Nilsen



CustomerNilsen
2

78

78



SQL FUNCTIONS

SQL COUNT() Function

SELECT COUNT(column_name) FROM table_name

SQL COUNT(DISTINCT column_name) Example

SELECT COUNT(DISTINCT Customer) AS NumberOfCustomers
FROM Orders

O_Id	OrderDate	OrderPrice	Customer
1	2008/11/12	1000	Hansen
2	2008/10/23	1600	Nilsen
3	2008/09/02	700	Hansen
4	2008/09/03	300	Hansen
5	2008/08/30	2000	Jensen
6	2008/10/04	100	Nilsen



NumberOfCustomers
3

79

79



SQL FUNCTIONS

SQL FIRST() Function

SELECT FIRST(column_name) FROM table_name

SQL FIRST() Example

SELECT FIRST(OrderPrice) AS FirstOrderPrice
FROM Orders

O_Id	OrderDate	OrderPrice	Customer
1	2008/11/12	1000	Hansen
2	2008/10/23	1600	Nilsen
3	2008/09/02	700	Hansen
4	2008/09/03	300	Hansen
5	2008/08/30	2000	Jensen
6	2008/10/04	100	Nilsen



FirstOrderPrice
1000

80

80



SQL FUNCTIONS

SQL LAST() Function

SELECT LAST(column_name) FROM table_name

SQL LAST() Example

*SELECT LAST(OrderPrice) AS LastOrderPrice
FROM Orders*

O_Id	OrderDate	OrderPrice	Customer
1	2008/11/12	1000	Hansen
2	2008/10/23	1600	Nilsen
3	2008/09/02	700	Hansen
4	2008/09/03	300	Hansen
5	2008/08/30	2000	Jensen
6	2008/10/04	100	Nilsen



LastOrderPrice
100

81

81



SQL FUNCTIONS

SQL MAX(), MIN() Function

SELECT MAX(column_name) FROM table_name

SELECT MIN(column_name) FROM table_name

SQL MAX(), MIN() Example

*SELECT MAX(OrderPrice) AS LargestOrderPrice
FROM Orders*

*SELECT MIN(OrderPrice) AS SmallestOrderPrice
FROM Orders*

O_Id	OrderDate	OrderPrice	Customer
1	2008/11/12	1000	Hansen
2	2008/10/23	1600	Nilsen
3	2008/09/02	700	Hansen
4	2008/09/03	300	Hansen
5	2008/08/30	2000	Jensen
6	2008/10/04	100	Nilsen



LargestOrderPrice	SmallestOrderPrice
2000	100

82

82



SQL FUNCTIONS

SQL SUM Function

SELECT SUM(column_name) FROM table_name

SQL SUM() Example

*SELECT SUM(OrderPrice) AS OrderTotal
FROM Orders*

O_Id	OrderDate	OrderPrice	Customer
1	2008/11/12	1000	Hansen
2	2008/10/23	1600	Nilsen
3	2008/09/02	700	Hansen
4	2008/09/03	300	Hansen
5	2008/08/30	2000	Jensen
6	2008/10/04	100	Nilsen



OrderTotal
5700

83

83



SQL FUNCTIONS

SQL MID() Function

SELECT MID(column_name,start[,length]) FROM table_name

Parameter	Description
column_name	Required. The field to extract characters from
Start	Required. Specifies the starting position (starts at 1)
Length	Optional. The number of characters to return. If omitted, the MID() function returns the rest of the text

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger



SmallCity
Sand
Sand
Stav

SQL MID() Example

*SELECT MID(City,1,4) as SmallCity
FROM Persons*

84

84



SQL FUNCTIONS

SQL LEN() Function

SELECT LEN (column_name,start[,length]) FROM table_name

SQL LEN () Example

*SELECT LEN(Address) as LengthOfAddress
FROM Persons*

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger



LengthOfAddress
12
9
9

85

85



SQL FUNCTIONS

SQL ROUND() Function

SELECT ROUND(column_name,decimals) FROM table_name

SQL ROUND() Example

*SELECT ProductName, ROUND(UnitPrice,0) as UnitPrice
FROM Products*

Prod_Id	ProductName	Unit	UnitPrice
1	Jarlsberg	1000 g	10.45
2	Mascarpone	1000 g	32.56
3	Gorgonzola	1000 g	15.67



ProductName	UnitPrice
Jarlsberg	10
Mascarpone	33
Gorgonzola	16

86

86



SQL FUNCTIONS

SQL NOW() Function

SELECT NOW() FROM table_name

SQL NOW() Example

*SELECT ProductName, UnitPrice, Now() as PerDate
FROM Products*

Prod_Id	ProductName	Unit	UnitPrice
1	Jarlsberg	1000 g	10.45
2	Mascarpone	1000 g	32.56
3	Gorgonzola	1000 g	15.67



ProductName	UnitPrice	PerDate
Jarlsberg	10.45	10/7/2008 11:25:02 AM
Mascarpone	32.56	10/7/2008 11:25:02 AM
Gorgonzola	15.67	10/7/2008 11:25:02 AM

87

87



SQL GROUP BY Statement

Aggregate functions often need an added GROUP BY statement

SQL GROUP BY Syntax

SELECT column_name, aggregate_function(column_name)

FROM table_name

WHERE column_name operator value

GROUP BY column_name

SQL GROUP BY Example

SELECT Customer, SUM(OrderPrice)

FROM Orders

GROUP BY Customer

O_Id	OrderDate	OrderPrice	Customer
1	2008/11/12	1000	Hansen
2	2008/10/23	1600	Nilson
3	2008/09/02	700	Hansen
4	2008/09/03	300	Hansen
5	2008/08/30	2000	Jensen
6	2008/10/04	100	Nilson



Customer	SUM(OrderPrice)
Hansen	2000
Nilson	1700
Jensen	2000

88

88



SQL GROUP BY Statement

GROUP BY More Than One Column

Example

```
SELECT Customer, OrderDate, SUM(OrderPrice)
FROM Orders
GROUP BY Customer, OrderDate
```

89

89



SQL HAVING CLAUSE

The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions.

SQL HAVING Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE column_name operator value
GROUP BY column_name
HAVING aggregate_function(column_name) operator value
```

90

90



SQL HAVING CLAUSE

SQL HAVING Example

```
SELECT Customer,SUM(OrderPrice)
FROM Orders
GROUP BY Customer
HAVING SUM(OrderPrice)<2000
```

O_Id	OrderDate	OrderPrice	Customer
1	2008/11/12	1000	Hansen
2	2008/10/23	1600	Nilsen
3	2008/09/02	700	Hansen
4	2008/09/03	300	Hansen
5	2008/08/30	2000	Jensen
6	2008/10/04	100	Nilsen



Customer	SUM(OrderPrice)
Nilsen	1700

91

91



SUB QUERY

Sub query syntax:

```
SELECT
  LASTNAME, FIRSTNAME
FROM EMPLOYEES
WHERE OFFICECODE IN (SELECT OFFICECODE
                     FROM OFFICES
                     WHERE COUNTRY = 'USA');
```

Note: IN, NOT IN, EXISTS, NOT EXISTS

92

92



SUB QUERY

Sub query example:

```
SELECT *
FROM PERSONS
WHERE P_ID NOT IN (SELECT P_ID
                   FROM orders)
```

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jakob	NULL	NULL

O_id	OrderNo	P_id
1	77895	3
2	44678	3
3	22456	2
4	22562	1



P_Id	LastName	FirstName	Address	City
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jakob	NULL	NULL

93

93



SUB QUERY

Sub query example:

Get all people who don't have any orders.

```
SELECT *
FROM PERSONS
WHERE P_ID NOT IN (SELECT P_ID
                   FROM orders)
```

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jakob	NULL	NULL

O_id	OrderNo	P_id
1	77895	3
2	44678	3
3	22456	2
4	22562	1



P_Id	LastName	FirstName	Address	City
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jakob	NULL	NULL

94

94



SUB QUERY

Sub query example:

Get all people who have any orders.

```
SELECT *
FROM PERSONS as p
WHERE exists (SELECT P_ID
               FROM orders as o
               WHERE o.P_id = p.P_id);
```

```
SELECT *
FROM PERSONS as p
WHERE p_id in (SELECT P_ID
               FROM orders );
```

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jakob	HULL	HULL

O_id	OrderNo	P_id
1	77895	3
2	44678	3
3	22456	2
4	22562	1



P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

95

95



STORE PROCEDURE

A stored procedure is a set of SQL statements that can be stored and executed on the database server. Stored procedures can help improve the performance, security, and maintainability of the database applications.

Create store procedure

```
CREATE PROCEDURE procedure_name ([parameter_list])
BEGIN
    procedure_body
END
```

Drop store procedure

```
Drop store Procedure_name
```

96

96



STORE PROCEDURE

Store procedure Example

```

delimiter //
create procedure GetPersonsInOrders()
begin
    SELECT *
    FROM PERSONS as p
    WHERE p_id in (SELECT P_ID
                  FROM orders);
End
//
call GetPersonsInOrders();

drop procedure GetPersonsInOrders;

```



P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

97

97



FUNCTION STATEMENT

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

The syntax the CREATE FUNCTION statement

```

CREATE FUNCTION function_Name(input_arguments)
statements
RETURNS output_parameter

```

98

98



FUNCTION STATEMENT

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

The syntax the CREATE FUNCTION statement

```
CREATE FUNCTION function_Name(input_arguments)
RETURNS output_parameter
DETERMINISTIC
statements
```

```
Drop function Function_Name;
```

99

99



FUNCTION STATEMENT

FUNCTION statement example:

Create function

```
delimiter //
create function CountOrders()
returns int
DETERMINISTIC
begin
    declare totalOrder int;
    select count(*) into totalOrder
    from orders;
    return totalOrder;
end//
```

call function

```
select CountOrders();
```



CountOrders()
4

Drop function

```
drop function CountOrders;
```

100

100



TRIGGER STATEMENT

A trigger is an object identified in the database and tied to an event occurring on a certain table.

These events include: INSERT, UPDATE or DELETE a table.

Trigger is executed automatically when a change action occurs in the table.

Users check data, synchronize data, ensure relationships between tables.

Syntax:

```
CREATE TRIGGER trigger_name trigger_time trigger_event
ON table_name
FOR EACH ROW
BEGIN
    Statements
END
```

Note: Trigger_time(Before,After), Trigger_event(Insert,update,delete)

Drop trigger syntax:

```
DROP TRIGGER table_name.trigger_name
```

101

101



TRIGGER STATEMENT

TRIGGER STATEMENT EXAMPLE:

```
delimiter //
create trigger priceIncrease
before insert on OrdersDetail
FOR EACH ROW
SET
    New.orderPrice = New.orderprice + New.orderprice *0.1;
//
```

```
insert into OrdersDetail values(9,'20240101',1000,'Tom');
```

```
drop trigger priceIncrease;
```



	O_id	OrderDate	OrderPrice	Customer
▶	1	2008-11-12	1000.00	Hansen
	2	2008-10-23	1600.00	Nilsen
	3	2008-08-02	700.00	Hansen
	4	2008-09-03	300.00	Hansen
	5	2008-08-30	2000.00	Jensen
	6	2008-10-04	100.00	Nilsen
	7	2024-01-01	100.00	Marry
	9	2024-01-01	1100.00	Tom

Note: NEW: contains new data lines, OLD: contains old data lines

102

102



SQL QUICK REFERENCE

SQL Statement	Syntax		
AND / OR	SELECT column_name(s) FROM table_name WHERE condition AND/OR condition	CREATE TABLE	CREATE TABLE table_name (column_name1 data_type, column_name2 data_type, column_name2 data_type, ...)
ALTER TABLE	ALTER TABLE table_name ADD column_name datatype or ALTER TABLE table_name DROP COLUMN column_name	CREATE INDEX	CREATE INDEX index_name ON table_name (column_name) or CREATE UNIQUE INDEX index_name ON table_name (column_name)
AS (alias)	SELECT column_name AS column_alias FROM table_name or SELECT column_name FROM table_name AS table_alias	CREATE VIEW	CREATE VIEW view_name AS SELECT column_name(s) FROM table_name WHERE condition
BETWEEN	SELECT column_name(s) FROM table_name WHERE column_name BETWEEN value1 AND value2	DELETE	DELETE FROM table_name WHERE some_column=some_value
CREATE DATABASE	CREATE DATABASE database_name		

103

103



SQL QUICK REFERENCE

DROP DATABASE	DROP DATABASE database_name	FULL JOIN	SELECT column_name(s) FROM table_name1 FULL JOIN table_name2 ON table_name1.column_name=table_name2.column_name
DROP INDEX	DROP INDEX table_name.index_name (SQL Server) DROP INDEX index_name ON table_name (MS Access) DROP INDEX index_name (DB2/Oracle) ALTER TABLE table_name DROP INDEX index_name (MySQL)	LIKE	SELECT column_name(s) FROM table_name WHERE column_name LIKE pattern
DROP TABLE	DROP TABLE table_name	ORDER BY	SELECT column_name(s) FROM table_name ORDER BY column_name [ASC DESC]
GROUP BY	SELECT column_name, aggregate_function(column_name) FROM table_name WHERE column_name operator value GROUP BY column_name	SELECT	SELECT column_name(s) FROM table_name
HAVING	SELECT column_name, aggregate_function(column_name) FROM table_name WHERE column_name operator value GROUP BY column_name HAVING aggregate_function(column_name) operator value	SELECT *	SELECT * FROM table_name
IN	SELECT column_name(s) FROM table_name WHERE column_name IN (value1,value2,...)	SELECT DISTINCT	SELECT DISTINCT column_name(s) FROM table_name
INSERT INTO	INSERT INTO table_name VALUES (value1, value2, value3,...) or INSERT INTO table_name (column1, column2, column3,...) VALUES (value1, value2, value3,...)	SELECT INTO	SELECT * INTO new_table_name [IN externaldatabase] FROM old_table_name or SELECT column_name(s) INTO new_table_name [IN externaldatabase] FROM old_table_name
INNER JOIN	SELECT column_name(s) FROM table_name1 INNER JOIN table_name2 ON table_name1.column_name=table_name2.column_name	SELECT TOP	SELECT TOP number percent column_name(s) FROM table_name
LEFT JOIN	SELECT column_name(s) FROM table_name1 LEFT JOIN table_name2 ON table_name1.column_name=table_name2.column_name	TRUNCATE TABLE	TRUNCATE TABLE table_name
RIGHT JOIN	SELECT column_name(s) FROM table_name1 RIGHT JOIN table_name2 ON table_name1.column_name=table_name2.column_name	UNION	SELECT column_name(s) FROM table_name1 UNION SELECT column_name(s) FROM table_name2
		UNION ALL	SELECT column_name(s) FROM table_name1 UNION ALL SELECT column_name(s) FROM table_name2
		UPDATE	UPDATE table_name SET column1=value, column2=value... WHERE some_column=some_value
		WHERE	SELECT column_name(s) FROM table_name WHERE column_name operator value

104

104

Q&A



105